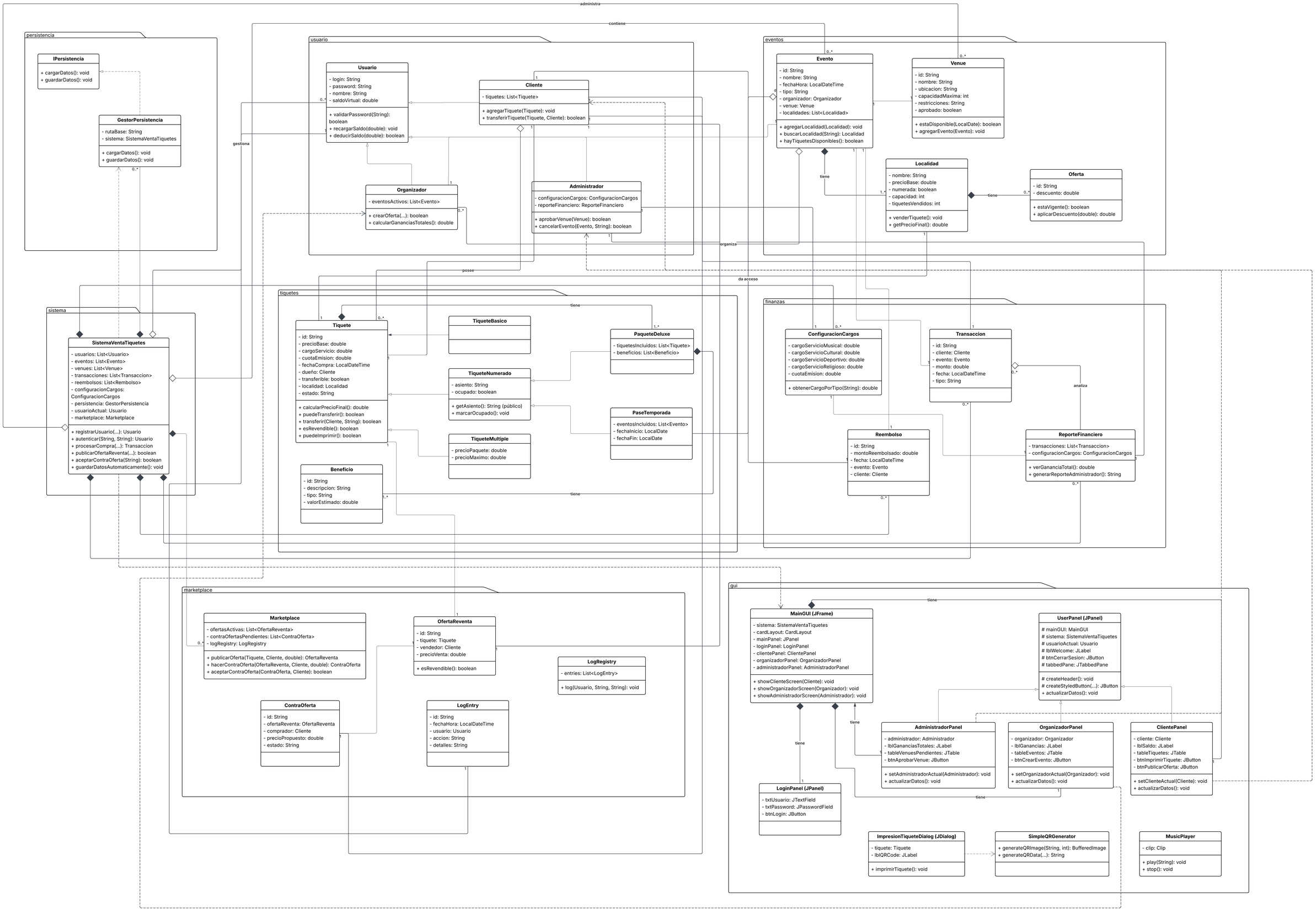

persistencia
IPersistencia
+ cargarDatos(): void
+ guardarDatos(): void
GestorPersistencia
- rutaBase: String
- sistema: SistemaVentaTiquetes
+ cargarDatos(): void
+ guardarDatos(): void
usuario
Usuario
- login: String
- password: String
- nombre: String
- saldoVirtual: double
+ validarPassword(String): boolean
+ recargarSaldo(double): void
+ deducirSaldo(double): boolean
Cliente
- tiquetes: List<Tiquete>
+ agregarTiquete(Tiquete): void
+ transferirTiquete(Tiquete, Cliente): boolean
Organizador
- eventosActivos: List<Evento>
+ crearOferta(...): boolean
+ calcularGananciasTotales(): double
Administrador
- configuracionCargos: ConfiguracionCargos
- reporteFinanciero: ReporteFinanciero
+ aprobarVenue(Venue): boolean
+ cancelarEvento(Evento, String): boolean
eventos
Evento
- id: String
- nombre: String
- fechaHora: LocalDateTime
- tipo: String
- organizador: Organizador
- venue: Venue
- localidades: List<Localidad>
+ agregarLocalidad(Localidad): void
+ buscarLocalidad(String): Localidad
+ hayTiquetesDisponibles(): boolean
Venue
- id: String
- nombre: String
- ubicacion: String
- capacidadMaxima: int
- restricciones: String
- aprobado: boolean
+ estaDisponible(LocalDate): boolean
+ agregarEvento(Evento): void
Localidad
- nombre: String
- precioBase: double
- numerada: boolean
- capacidad: int
- tiquetesVendidos: int
+ venderTiquete(): void
+ getPrecioFinal(): double
Oferta
- id: String
- descuento: double
+ estaVigente(): boolean
+ aplicarDescuento(double): double
sistema
SistemaVentaTiquetes
- usuarios: List<Usuario>
- eventos: List<Evento>
- venues: List<Venue>
- transacciones: List<Transaccion>
- reembolsos: List<Rembolso>
- configuracionCargos: ConfiguracionCargos
- persistencia: GestorPersistencia
- usuarioActual: Usuario
- marketplace: Marketplace
+ registrarUsuario(...): Usuario
+ autenticar(String, String): Usuario
+ procesarCompra(...): Transaccion
+ publicarOfertaReventa(...): boolean
+ aceptarContraOferta(String): boolean
+ guardarDatosAutomaticamente(): void
tiquetes
Tiquete
- id: String
- precioBase: double
- cargoServicio: double
- cuotaEmision: double
- fechaCompra: LocalDateTime
- dueño: Cliente
- transferible: boolean
- localidad: Localidad
- estado: String
+ calcularPrecioFinal(): double
+ puedeTransferir(): boolean
+ transferir(Cliente, String): boolean
+ esRevendible(): boolean
+ puedeImprimir(): boolean
TiqueteBasico
TiqueteNumerado
- asiento: String
- ocupado: boolean
+ getAsiento(): String (público)
+ marcarOcupado(): void
TiqueteMultiple
- precioPaquete: double
- precioMaximo: double
PaqueteDeluxe
- tiquetesIncluidos: List<Tiquete>
- beneficios: List<Beneficio>
PaseTemporada
- eventosIncluidos: List<Evento>
- fechaInicio: LocalDate
- fechaFin: LocalDate
Beneficio
- id: String
- descripcion: String
- tipo: String
- valorEstimado: double
finanzas
ConfiguracionCargos
- cargoServicioMusical: double
- cargoServicioCultural: double
- cargoServicioDeportivo: double
- cargoServicioReligioso: double
- cuotaEmision: double
+ obtenerCargoPorTipo(String): double
Transaccion
- id: String
- cliente: Cliente
- evento: Evento
- monto: double
- fecha: LocalDateTime
- tipo: String
Reembolso
- id: String
- montoReembolsado: double
- fecha: LocalDateTime
- evento: Evento
- cliente: Cliente
ReporteFinanciero
- transacciones: List<Transaccion>
- configuracionCargos: ConfiguracionCargos
+ verGananciaTotal(): double
+ generarReporteAdministrador(): String
marketplace
Marketplace
- ofertasActivas: List<OfertaReventa>
- contraOfertasPendientes: List<ContraOferta>
- logRegistry: LogRegistry
+ publicarOferta(Tiquete, Cliente, double): OfertaReventa
+ hacerContraOferta(OfertaReventa, Cliente, double): ContraOferta
+ aceptarContraOferta(ContraOferta, Cliente): boolean
OfertaReventa
- id: String
- tiquete: Tiquete
- vendedor: Cliente
- precioVenta: double
+ esRevendible(): boolean
ContraOferta
- id: String
- ofertaReventa: OfertaReventa
- comprador: Cliente
- precioPropuesto: double
- estado: String
LogRegistry
- entries: List<LogEntry>
+ log(Usuario, String, String): void
LogEntry
- id: String
- fechaHora: LocalDateTime
- usuario: Usuario
- accion: String
- detalles: String
gui
MainGUI (JFrame)
- sistema: SistemaVentaTiquetes
- cardLayout: CardLayout
- mainPanel: JPanel
- loginPanel: LoginPanel
- clientePanel: ClientePanel
- organizadorPanel: OrganizadorPanel
- administradorPanel: AdministradorPanel
+ showClienteScreen(Cliente): void
+ showOrganizadorScreen(Organizador): void
+ showAdministradorScreen(Administrador): void
UserPanel (JPanel)
mainGUI: MainGUI
sistema: SistemaVentaTiquetes
usuarioActual: Usuario
lblWelcome: JLabel
btnCerrarSesion: JButton
tabbedPane: JTabbedPane
createHeader(): void
createStyledButton(...): JButton
+ actualizarDatos(): void
LoginPanel (JPanel)
- txtUsuario: JTextField
- txtPassword: JPasswordField
- btnLogin: JButton
AdministradorPanel
- administrador: Administrador
- lblGananciasTotales: JLabel
- tableVenuesPendientes: JTable
- btnAprobarVenue: JButton
+ setAdministradorActual(Administrador): void
+ actualizarDatos(): void
OrganizadorPanel
- organizador: Organizador
- lblGanancias: JLabel
- tableEventos: JTable
- btnCrearEvento: JButton
+ setOrganizadorActual(Organizador): void
+ actualizarDatos(): void
ClientePanel
- cliente: Cliente
- lblSaldo: JLabel
- tableTiquetes: JTable
- btnImprimirTiquete: JButton
- btnPublicarOferta: JButton
+ setClienteActual(Cliente): void
+ actualizarDatos(): void
ImpresionTiqueteDialog (JDialog)
- tiquete: Tiquete
- lblQRCode: JLabel
+ imprimirTiquete(): void
SimpleQRGenerator
+ generateQRImage(String, int): BufferedImage
+ generateQRData(...): String
MusicPlayer
- clip: Clip
+ play(String): void
+ stop(): void
administra
contiene
gestiona
posee
organiza
da acceso
tiene
analiza